



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"МИРЭА - Российский технологический университет"

РТУ МИРЭА

Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения

**ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №6
по дисциплине
«Технологии виртуализации клиент-серверных приложений»**

Выполнил студент группы ИКБО-16-22

Грибков А.С.

Принял преподаватель кафедры ИиППО

Волков М.Ю.

Практические работы выполнены

«__»_____2025 г.

«Зачтено»

«__»_____2025 г.

Москва
2025

СОДЕРЖАНИЕ

1 Цель работы	3
2 Ход работы	4
2.1 Создание Deployment для frontend	4
2.2 Создание Service для frontend	4
2.3 Создание ConfigMap	5
2.4 Создание Secret для Redis	6
2.5 Развертывание Redis StatefulSet	7
2.6 Создание Services для Redis	8
2.7 Развертывание FileServer	9
2.8 Настройка Ingress	10
2.9 Проверка работоспособности	12
3 Вывод	16
4 Ответы на вопросы	17
5 Список использованной литературы	19

1 ЦЕЛЬ РАБОТЫ

Изучить процесс развертывания распределенных приложений в Kubernetes с использованием различных типов ресурсов.

Задачи работы:

1. Развернуть stateless приложение (frontend) с использованием Deployment
2. Настроить конфигурацию приложения через ConfigMap
3. Обеспечить безопасность с помощью Secret
4. Развернуть stateful приложение (Redis) с использованием StatefulSet
5. Настроить репликацию Redis Master-Slave
6. Создать Service для доступа к приложениям
7. Настроить маршрутизацию HTTP-трафика через Ingress
8. Развернуть статический файловый сервер (NGINX)
9. Проверить работоспособность всей системы

2 ХОД РАБОТЫ

2.1 Создание Deployment для frontend

Deployment представляет собой декларативный способ управления репликацией подов. Для клиентского приложения создан ресурс Deployment с двумя репликами.

Листинг 2.1 — Манифест frontend.yaml

```
apiVersion: apps/v1 kind: Deployment metadata: labels: app: frontend name: frontend namespace: default spec: replicas: 2 selector: matchLabels: app: frontend template: metadata: labels: app: frontend spec: containers: - image: gribkov/journal-server:v1 imagePullPolicy: Never name: frontend env: - name: JOURNAL_ENTRIES valueFrom: configMapKeyRef: name: frontend-config key: journalEntries volumeMounts: - name: passwd-volume readOnly: true mountPath: "/etc/redis-passwd" resources: requests: cpu: "100m" memory: "256Mi" limits: cpu: "500m" memory: "512Mi" volumes: - name: passwd-volume secret: secretName: redis-passwd
```

В манифесте установлены requests и limits для ресурсов CPU и памяти. Это обеспечивает гарантированное выделение ресурсов и ограничение их потребления.

```
alexgribkov@i113971732 practical-work-6 % kubectl get deployment frontend
NAME          READY    UP-TO-DATE    AVAILABLE    AGE
frontend      2/2      2             2            82m
```

Рисунок 2.1 — Применение Deployment для frontend

2.2 Создание Service для frontend

Service обеспечивает стабильную точку доступа к подам Deployment через балансировку нагрузки.

Листинг 2.2 — Манифест traffic-service.yml для frontend

```
apiVersion: v1 kind: Service metadata: labels: app: frontend name: frontend namespace: default spec: ports: - port: 8080 protocol: TCP targetPort: 8080 selector: app: frontend type: ClusterIP
```

Service типа ClusterIP делает приложение доступным только внутри кластера. Связь с подами осуществляется через метку app: frontend.

```
alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/frontend/traffic-service.yml
service/frontend unchanged
```

```
alexgribkov@i113971732 practical-work-6 % kubectl get svc frontend
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
frontend	ClusterIP	10.101.11.165	<none>	8080/TCP	72m

Рисунок 2.2 — Создание Service для frontend и проверка его состояния

2.3 Создание ConfigMap

ConfigMap используется для хранения конфигурационных данных в формате ключ-значение. В данном случае создается ConfigMap с параметром journalEntries.

Листинг 2.3 — Команда создания ConfigMap

```
kubectl create configmap frontend-config --from-literal=journalEntries=10
```

Эта конфигурация передается в контейнер как переменная среды JOURNAL_ENTRIES, что видно в манифесте Deployment.

```
alexgribkov@i113971732 practical-work-6 % kubectl get configmap frontend-config -o yaml

apiVersion: v1
data:
  journalEntries: "10"
kind: ConfigMap
metadata:
  creationTimestamp: "2025-11-18T16:56:20Z"
  name: frontend-config
  namespace: default
  resourceVersion: "1028"
  uid: afe66103-38f0-43b2-aed9-c82521d27a77
```

Рисунок 2.3 — Создание ConfigMap и просмотр его содержимого

2.4 Создание Secret для Redis

Secret используется для хранения конфиденциальных данных, таких как пароли. Пароль для Redis генерируется случайным образом и сохраняется в Secret.

Листинг 2.4 — Команда создания Secret с паролем Redis

```
kubectl create secret generic redis-passwd --from-literal=password=$(openssl rand -base64 32)
```

Secret монтируется в контейнеры как том в каталог `/etc/redis-passwd`. Том создается в виде файловой системы `tmpfs` в оперативной памяти для повышения безопасности.

```
alexgribkov@i113971732 practical-work-6 % kubectl get secret redis-passwd

NAME          TYPE      DATA   AGE
redis-passwd  Opaque    1       73m
```

Рисунок 2.4 — Создание Secret для хранения пароля Redis

2.5 Развертывание Redis StatefulSet

StatefulSet используется для развертывания stateful приложений с согласованными именами подов и постоянными томами. Redis развертывается с тремя репликами: одна master и две slave.

Перед развертыванием Redis необходимо создать ConfigMap с скриптом launch.sh для настройки репликации.

Листинг 2.5 — Скрипт launch.sh для настройки Master-Slave репликации

```
#!/bin/sh
PASSWORD=(cat/etc/redis - passwd/passwd)HOSTNAME=(echo ${HOSTNAME})
if [ "${HOSTNAME}" = "redis-0" ]; then          redis-server --
requirepass "${PASSWORD}" else redis-server -slaveof redis-0.redis
6379 -masterauth "${PASSWORD}" --requirepass "${PASSWORD}" fi
```

Скрипт проверяет имя хоста: если это redis-0, запускается master, иначе запускается slave с подключением к redis-0.

Листинг 2.6 — Команда создания ConfigMap со скриптом

```
kubectl create configmap redis-config --from-
file=launch.sh=app/redis/launch.sh
```

```
alexgribkov@i113971732 practical-work-6 % kubectl get configmap redis-
config
NAME          DATA   AGE
redis-config  1       73m
```

Рисунок 2.5 — Создание ConfigMap с launch.sh скриптом

Листинг 2.7 — Манифест redis.yml

```
apiVersion: apps/v1 kind: StatefulSet metadata: name: redis spec:
serviceName: redis replicas: 3 selector: matchLabels: app: redis
template: metadata: labels: app: redis spec: containers: - name:
redis image: redis:5-alpine ports: - containerPort: 6379 name:
redis volumeMounts: - name: data mountPath: /data - name: script
mountPath: /script/launch.sh subPath: launch.sh - name: passwd-
volume mountPath: /etc/redis-passwd command: - sh - -c -
/script/launch.sh volumes: - name: script configMap: name: redis-
config defaultMode: 0777 - name: passwd-volume secret: secretName:
redis-passwd volumeClaimTemplates: - metadata: name: data spec:
accessModes: [ "ReadWriteOnce" ] resources: requests: storage:
10Gi
```

StatefulSet создает три пода с именами redis-0, redis-1, redis-2. Для каждого пода автоматически создается PersistentVolumeClaim размером 10Gi.

```
alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/redis/redis.yml
statefulset.apps/redis configured

alexgribkov@i113971732 practical-work-6 % kubectl get statefulset redis
NAME      READY   AGE
redis     3/3     74m
```

Рисунок 2.6 — Применение StatefulSet для Redis

```
alexgribkov@i113971732 practical-work-6 % kubectl get pvc
NAME      STATUS   VOLUME                                     CAP
data-redis-0 Bound    pvc-43ae3ad1-2dce-44cb-bfbe-86e870f43eb4 10G
i         RWO
data-redis-1 Bound    pvc-1b8e165e-f723-4fc5-a5d7-9c3a784e2c15 10G
i         RWO
data-redis-2 Bound    pvc-e05aa539-f07a-4360-afaa-4162e2e76589 10G
i         RWO
```

Рисунок 2.7 — Список PersistentVolumeClaim для подов Redis

2.6 Создание Services для Redis

Для Redis создаются два Service: один для чтения данных и headless Service для записи.

Листинг 2.8 — Service для чтения (redis-read.yml)

```
apiVersion: v1 kind: Service metadata: labels: app: redis name:
redis namespace: default spec: ports: - port: 6379 protocol: TCP
targetPort: 6379 selector: app: redis type: ClusterIP
```

Этот Service балансирует запросы на чтение между всеми тремя репликами Redis.

Листинг 2.9 — Headless Service для записи (redis-write.yml)

```
apiVersion: v1 kind: Service metadata: labels: app: redis-write
name: redis-write spec: clusterIP: None ports: - port: 6379
selector: app: redis
```


Headless Service (clusterIP: None) создает отдельную DNS-запись для каждого пода. Для записи используется адрес redis-0.redis-write, который указывает на master.

```
alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/redis/r
edis-read.yml
service/redis unchanged

alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/redis/r
edis-write.yml
service/redis-write unchanged

alexgribkov@i113971732 practical-work-6 % kubectl get svc | grep redis
```

redis	ClusterIP	10.103.105.195	<none>	6379/TCP	75
m					
redis-write	ClusterIP	None	<none>	6379/TCP	75
m					

Рисунок 2.8 — Список всех Service для Redis в кластере

2.7 Развертывание FileServer

FileServer представляет собой статический веб-сервер на базе NGINX для раздачи HTML, CSS и JavaScript файлов.

Листинг 2.10 — Манифест fileserver.yml

```
apiVersion: apps/v1 kind: Deployment metadata: labels: app:
fileserver name: fileserver namespace: default spec: replicas: 2
selector: matchLabels: app: fileserver template: metadata: labels:
app: fileserver spec: containers: - image: gribkov/static-files:v1
imagePullPolicy: Never name: fileserver resources: requests: cpu:
"50m" memory: "128Mi" limits: cpu: "200m" memory: "256Mi"
```

Листинг 2.11 — Service для fileserver (fileserver-service.yml)

```
apiVersion: v1 kind: Service metadata: labels: app: fileserver
name: fileserver namespace: default spec: ports: - port: 80
protocol: TCP targetPort: 80 selector: app: fileserver type:
ClusterIP
```

```
alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/fileserver/fileserver.yml
```

```
deployment.apps/fileserver unchanged
```

```
alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/fileserver/fileserver-service.yml
```

```
service/fileserver unchanged
```

```
alexgribkov@i113971732 practical-work-6 % kubectl get deployment fileserver
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
fileserver	2/2	2	2	75m

```
alexgribkov@i113971732 practical-work-6 % kubectl get svc fileserver
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
fileserver	ClusterIP	10.111.113.196	<none>	80/TCP	75m

Рисунок 2.9 — Развертывание FileServer и создание Service

2.8 Настройка Ingress

Ingress обеспечивает маршрутизацию внешнего HTTP-трафика к Service внутри кластера. Перед созданием Ingress необходимо включить соответствующий addon в Minikube.

Листинг 2.12 — Команда включения Ingress addon

```
minikube addons enable ingress
```

```
alexgribkov@i113971732 practical-work-6 % minikube addons enable ingress
💡 ingress is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
💡 After the addon is enabled, please run "minikube tunnel" and your ingress resources would be available at "127.0.0.1"
  ▪ Using image registry.k8s.io/ingress-nginx/controller:v1.13.2
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
  ▪ Using image registry.k8s.io/ingress-nginx/kube-webhook-certgen:v1.6.2
🔍 Verifying ingress addon...
🌟 The 'ingress' addon is enabled
```

Рисунок 2.10 — Включение Ingress addon в Minikube

Листинг 2.13 — Манифест ingress.yaml

```
apiVersion: networking.k8s.io/v1 kind: Ingress metadata: name: frontend-ingress spec: rules: - http: paths: - path: /api pathType: Prefix backend: service: name: frontend port: number: 8080 - path: / pathType: Prefix backend: service: name: fileserver port: number: 80
```

Ingress настроен на маршрутизацию запросов: путь /api направляется на frontend Service (порт 8080), а путь / на fileserver Service (порт 80). Важно, что правило для /api должно идти перед правилом для /, иначе все запросы будут перехватываться fileserver.

```
alexgribkov@i113971732 practical-work-6 % kubectl apply -f app/frontend/ingress.yaml
ingress.networking.k8s.io/frontend-ingress unchanged

alexgribkov@i113971732 practical-work-6 % kubectl get ingress
NAME                CLASS    HOSTS    ADDRESS    PORTS    AGE
frontend-ingress    <none>   *        192.168.49.2    80      84m
```

Рисунок 2.11 — Применение Ingress и проверка его состояния

2.9 Проверка работоспособности

После развертывания всех компонентов необходимо проверить состояние кластера и работоспособность приложения.

```
alexgribkov@i113971732 practical-work-6 % kubectl get all
```

NAME	READY	STATUS	RESTARTS	AGE
pod/fileserver-75576c7b6c-7tzxs	1/1	Running	0	90m
pod/fileserver-75576c7b6c-xxtsv	1/1	Running	0	90m
pod/frontend-667d95dc97-5p2mv	1/1	Running	0	102m
pod/frontend-667d95dc97-6jp8c	1/1	Running	0	102m
pod/redis-0	1/1	Running	0	90m
pod/redis-1	1/1	Running	0	90m
pod/redis-2	1/1	Running	0	90m

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)
service/fileserver	ClusterIP	10.111.113.196	<none>	80/TCP
service/frontend	ClusterIP	10.101.11.165	<none>	8080/TCP
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP
service/redis	ClusterIP	10.103.105.195	<none>	6379/TCP
service/redis-write	ClusterIP	None	<none>	6379/TCP

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/fileserver	2/2	2	2	90m
deployment.apps/frontend	2/2	2	2	102m

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/fileserver-75576c7b6c	2	2	2	90m
replicaset.apps/frontend-667d95dc97	2	2	2	102m

NAME	READY	AGE
statefulset.apps/redis	3/3	90m

Рисунок 2.12 — Список всех ресурсов в кластере Kubernetes


```
alexgribkov@i113971732 practical-work-6 % kubectl exec redis-1 -- sh -
c 'redis-cli -a $(cat /etc/redis-passwd/passwd) INFO replication'
Warning: Using a password with '-a' or '-u' option on the command line
interface may not be safe.
# Replication
role:slave
master_host:redis-0.redis
master_port:6379
master_link_status:up
master_last_io_seconds_ago:8
master_sync_in_progress:0
slave_repl_offset:3038
slave_priority:100
slave_read_only:1
connected_slaves:0
master_replid:6515b86e02f5b2c6e4c9e518b1dbb82a1efead8b
master_replid2:0000000000000000000000000000000000000000000000000000000000000000
master_repl_offset:3038
second_repl_offset:-1
repl_backlog_active:1
repl_backlog_size:1048576
repl_backlog_first_byte_offset:1
repl_backlog_histlen:3038
```

Рисунок 2.14 — Проверка роли Redis Slave (redis-1)

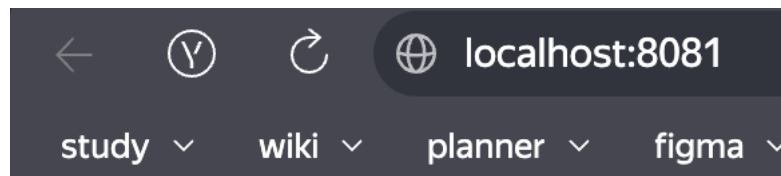
Для проверки работы приложения через браузер необходимо настроить port-forward или использовать Ingress.

```
alexgribkov@i113971732 practical-work-6 % curl http://localhost:8080/a
pi
[{"message":"Test from report"}]%
```

Рисунок 2.15 — GET-запрос к API для получения списка записей журнала

```
alexgribkov@i113971732 practical-work-6 % curl -X POST http://localhos
t:8080/api \
-H "Content-Type: application/json" \
-d '{"message": "Test from report"}'
[{"message":"Test from report"}]%
```

Рисунок 2.16 — POST-запрос к API для добавления новой записи



My Static App

Рисунок 2.17 — Доступ к статическому серверу через браузер

3 ВЫВОД

В результате выполнения практической работы было успешно развернуто распределенное приложение в Kubernetes с использованием Minikube. Достигнуты следующие результаты:

1. Развернуто stateless приложение frontend с использованием Deployment и двумя репликами
2. Настроена конфигурация приложения через ConfigMap для передачи параметра journalEntries
3. Обеспечена безопасность хранения пароля Redis с помощью Secret
4. Развернуто stateful приложение Redis с использованием StatefulSet и тремя репликами
5. Настроена автоматическая репликация Redis Master-Slave через launch.sh скрипт
6. Созданы Service различных типов для обеспечения сетевого доступа к приложениям
7. Настроен Ingress для маршрутизации HTTP-трафика по путям /api и /
8. Развернут статический файловый сервер на базе NGINX
9. Проверена работоспособность всей системы через браузер

Работа продемонстрировала ключевые концепции Kubernetes: декларативное управление ресурсами, автоматическая репликация и масштабирование, управление конфигурацией и секретами, работа с постоянными томами для stateful приложений.

4 ОТВЕТЫ НА ВОПРОСЫ

1. Для чего нужен ресурс Deployment?

Deployment используется для декларативного управления репликацией подов. Он обеспечивает автоматическое создание заданного количества идентичных реплик приложения, поддерживает механизмы поэтапного обновления (rolling updates), откат к предыдущим версиям, а также автоматическое восстановление подов в случае сбоев. Deployment сочетает возможности ReplicaSet с системой управления версиями.

2. Почему не стоит хранить пароли в ConfigMap?

ConfigMap предназначен для хранения обычных конфигурационных данных и не обеспечивает дополнительной защиты. Данные в ConfigMap хранятся в открытом виде и легко доступны через kubectl. Для конфиденциальных данных следует использовать Secret, который обеспечивает кодирование данных в base64 и может монтироваться как tmpfs том в памяти, не записываясь на диск.

3. Что необходимо для настройки внешнего доступа для HTTP-трафика? Назовите шаги.

Шаги настройки внешнего доступа:

1. Создать Service типа ClusterIP для внутренней балансировки нагрузки между подами
2. Установить и настроить Ingress Controller в кластере (например, через minikube addons enable ingress)
3. Создать ресурс Ingress с правилами маршрутизации, указывающими соответствие между HTTP-путями и Service
4. Настроить DNS или использовать IP-адрес Ingress Controller для внешнего доступа
4. Чем отличается разворачивание stateful от разворачивания клиентского приложения?

Stateful приложения используют StatefulSet вместо Deployment. Основные отличия: StatefulSet обеспечивает согласованные имена подов (redis-0, redis-1, redis-2), определенный порядок создания и удаления подов, возможность использования PersistentVolumeClaim для каждого пода индивидуально. Клиентские приложения обычно stateless, используют Deployment с произвольными именами подов и не требуют постоянного хранилища.

5. Где хранятся том с секретными данными?

Том с секретными данными создается в виде файловой системы tmpfs в оперативной памяти контейнера. Данные не записываются на диск, что повышает безопасность. Том монтируется в указанный каталог контейнера (например, /etc/redis-passwd) и доступен только для чтения.

6. Как работает связка PersistentVolume и PersistentVolumeClaim?

PersistentVolume (PV) представляет физический том хранилища в кластере. PersistentVolumeClaim (PVC) является запросом на выделение хранилища с определенными характеристиками (размер, режим доступа). Kubernetes автоматически связывает PVC с подходящим PV. В случае StatefulSet для каждого пода создается отдельный PVC через volumeClaimTemplates, что обеспечивает изолированное хранилище для каждой реплики stateful приложения.

5 СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. Kubernetes: лучшие практики. — СПб.: Питер, 2021. — 288 с.: ил. — (Серия «Для профессионалов»).
2. K8S для начинающих. Первая часть — Текст: электронный [сайт]. — URL: <https://habr.com/ru/post/589415/> (дата обращения: 17.11.2025).
3. Kubernetes или с чего начать, чтобы понять что это и зачем он нужен — Текст: электронный [сайт]. — URL: <https://habr.com/ru/company/otus/blog/537162/> (дата обращения: 17.11.2025).
4. Основы Kubernetes — Текст: электронный [сайт]. — URL: <https://habr.com/ru/post/258443/> (дата обращения: 17.11.2025).